These macros specify whether the atomic types corresponding to particular built-in types are lock-free or not.

Macro declarations

```
#define ATOMIC_BOOL_LOCK_FREE  see description
#define ATOMIC_CHAR_LOCK_FREE  see description
#define ATOMIC_SHORT_LOCK_FREE  see description
#define ATOMIC_INT_LOCK_FREE see description
#define ATOMIC_LONG_LOCK_FREE  see description
#define ATOMIC_LLONG_LOCK_FREE  see description
#define ATOMIC_CHAR16_T_LOCK_FREE  see description
#define ATOMIC_CHAR32_T_LOCK_FREE  see description
#define ATOMIC_WCHAR_T_LOCK_FREE  see description
#define ATOMIC_POINTER_LOCK_FREE  see description
```

The value of `ATOMIC_`*xxx*`_LOCK_FREE` is either 0, 1, or 2. A value of 0 means that operations on both the signed and unsigned atomic types corresponding to the named type are never lock-free, a value of 1 means that the operations may be lock-free for particular instances of those types and not for others, and a value of 2 means that the operations are always lock-free. For example, if `ATOMIC_INT_LOCK_FREE` is 2, operations on instances of `std::atomic<int>` and `std::atomic<unsigned>` are always lock-free.

The macro `ATOMIC_POINTER_LOCK_FREE` describes the lock-free property of operations on the atomic pointer specializations `std::atomic<T*>`.

D.3.3. ATOMIC_VAR_INIT macro

The `ATOMIC_VAR_INIT` macro provides a means of initializing an atomic variable to a particular value.

Declaration

```
#define ATOMIC_VAR_INIT(value) see description
```